IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN XXXX-XXXX, Vol. 0, No. 0, pp. 1–23. DOI:XXXXXXXX

Fault Attack Countermeasures for SPHINCS+: WOTS+ with Checksum Segmentation and Subtree Root Children Caching

Yan Li[1], Tianlong Yan[1], Gechen Liu[1], Shuang Yao[2], Ningning He[2], Hao Wen[1] and Yiming Xue[1*]

[1] China Agricultural University, Beijing, China
liyan_00@cau.edu.cn, tianlong_yan@cau.edu.cn, s20243081898@cau.edu.cn, wenh06@cau.edu.cn, xueym@cau.edu.cn
[2] Beijing CEC Huada Electronic Design Co. Ltd, Beijing, China
yaoshuang@hed.com.cn, henn@hed.com.cn

Abstract. SPHINCS+ is a stateless hash-based digital signature algorithm (SLH-DSA), standardized by NIST (FIPS 205). However, it is vulnerable to fault attacks, especially in embedded and IoT systems where devices are often more accessible to attackers. Additionally, existing countermeasures against fault attacks on SPHINCS+ generally require either additional hardware resources, substantial memory capacity, or redundant computations.
The fault attacks in SPHINCS+ arise from the reuse of a WOTS+ instance, leading to security degradation and signature forgery. Prior research on this security degradation was limited to inaccurate probabilistic assumptions or slow computations.
In this paper, we introduce a novel analytical method based on Generating Functions to efficiently and precisely compute the post-reuse security of WOTS+. Through this rigorous analysis, we discover that the security degradation follows a bimodal distribution, and the key to improving the security floor lies in the number of possible values for the checksum's most significant block. Based on this insight, we propose WOTS+ with Checksum Segmentation (WOTS+CS), the first enhancement of the WOTS+ designed to improve post-reuse security (e.g., raising the security floor from 34.9 to 49.8 bits in the 256-bit instance) by structurally eliminating the weak instances. To strengthen protection further, we introduce the Subtree Root Children Caching (SRCC) scheme, which validates the subtree root to prevent WOTS+ from signing an incorrect root. By caching the subtree roots of the top few layers, the storage overhead remains manageable.
Finally, we conducted fault injection experiments on SPHINCS+ using a ChipWhisperer to evaluate the difficulty of fault injection. Subsequently, we evaluated the effectiveness of our countermeasures in a complete fault attack, demonstrating that our methods increase the attacker's difficulty across all phases of fault attacks.

Keywords: SPHINCS+ · post-quantum cryptography · fault attack

1 Introduction

With the rapid development of quantum computers, the danger posed by quantum algorithms [Sho99, Gro96] to traditional public key cryptography is increasing. Researchers are dedicating more attention to creating a new generation of cryptographic algorithms capable of resisting quantum attacks. In 2017, the National Institute of Standards and Technology

*Corresponding author

Licensed under Creative Commons License CC-BY 4.0.

(NIST) started a standardization process [NIS16] for Post-Quantum Cryptography (PQC). During this period, SPHINCS [BHH+15] and SPHINCS+ [BHK+19] (an improvement of SPHINCS) emerged as strong candidates due to their robust security. In 2024, the Secretary of Commerce approved the Federal Information Processing Standard 205 (FIPS 205), the Stateless Hash-Based Digital Signature Algorithm (SLH-DSA) [NIS24] (the standardized version of SPHINCS+), as part of the post-quantum cryptography standards.

Despite its cryptographic robustness, SPHINCS+ is inherently vulnerable to fault attacks because it relies on the WOTS+ scheme. While WOTS+ keys are secure for a single use, fault injections can force a specific WOTS+ instance to sign two different messages (key reuse), causing its security to collapse [Gen17]. This vulnerability is particularly critical, especially in resource-constrained scenarios such as embedded and IoT systems. These scenarios are typically more exposed to attackers, increasing their vulnerability to fault injection attacks. The first fault attack against SPHINCS was proposed by Castelnovi et al. [CMP18]. In this attack, a fault was injected to force a WOTS+ key pair to sign an incorrect subtree root. The analysis in [Gen23] shows that a single-bit corruption in almost any computation has a catastrophic impact on the security guarantees of SPHINCS+.

Within a complete fault attack, the adversary can forge the signature for an arbitrary message. A complete fault attack targeting SPHINCS+ comprises three critical steps: **fault injection**, **signature acquisition**, and **signature forgery**.

In order to tackle fault injection, different countermeasures have been proposed. The method of redundancy checking [ALCZ20] works by recomputing the same signature several times and halting the signing if inconsistencies are found, leading to a substantial increase in signing time. Another study proposed "Impeccable Keccak" [GOW+24], a software-hardware implementation of SPHINCS+ that incorporates a SHA3 processor capable of concurrent data processing with redundant encoding, thereby facilitating error detection for a limited number of bit errors. However, this approach requires substantial hardware resources. Caching WOTS+ signature [Gen23], an example of storing intermediate values, mitigates fault attacks at the expense of memory consumption.

The signature forgery phase of fault attacks on SPHINCS+ fundamentally relies on the reuse of WOTS+. As an integral component of SPHINCS+, WOTS+ is designed as a one-time signature scheme. Consequently, its security diminishes upon reuse.

Groot Bruinderink et al. [GBH17] provided a formula for calculating the security level of WOTS+ after two uses. However, this formula was derived under the simplified assumption that the checksum is probabilistically independent of the message, introducing inaccuracies into their calculated results, as noted by Fluhrer et al. in [Flu23].

Fluhrer [Flu23] addressed this by calculating the probability distribution of WOTS+ security strength after two signatures are given. This method employed a combination of sampling and Dynamic Programming (DP). It sampled numerous pairs of WOTS+ signatures to approximate the distribution of the security strength, which was calculated using DP for each pair.

In this paper, we propose a fault attack countermeasure for SPHINCS+ by combining WOTS+CS and SRCC. Our contributions can be summarized as follows:

- **Formal Analysis of WOTS+ Post-Reuse Security via Generating Functions**: We introduce a rigorous mathematical framework using Generating Functions to calculate the exact post-reuse security of WOTS+. This method is faster than previous approaches based on Dynamic Programming [Flu23]. Through this analysis, we reveal a critical vulnerability: standard WOTS+ exhibits a bimodal security distribution under reuse, where the average security is heavily dragged down by "weak instances" caused by the checksum's most significant block value being limited to two possible states.

- **WOTS+CS, an Improved WOTS+:** Based on our analysis, we propose WOTS+

with Checksum Segmentation (WOTS+CS), the first improvement of WOTS+ designed to enhance post-reuse security. WOTS+CS modifies the checksum mechanism by dividing the message into segments, with each segment having its own checksum. This ensures the checksum's most significant value can take on multiple possible values, which increases the difficulty for an attacker to forge a signature. For instance, it raises the average post-reuse security strength from 35.4 bits to 49.8 bits in the 256-bit instantiation.

- **Practical Fault Attacks Countermeasures:** We propose a low-cost and practical defense methodology against the three phases of a fault attack.
  - For the fault injection phase, we introduce **Subtree Root Children Caching (SRCC)**, a low-overhead countermeasure. By caching the subtree roots in the top few layers, SRCC prevents WOTS+ from signing an incorrect root while requiring minimal storage and increasing the signing speed. This is an improvement over existing cache-based countermeasures.
  - We conducted fault injection experiments to validate the effectiveness of our proposed hybrid defense, which combines SRCC in higher layers and WOTS+CS in lower layers. The results confirm that our methods increase the attacker's difficulty across all attack phases.

The remainder of this paper is structured as follows. In Section 2, we review the SPHINCS+ scheme and the fundamentals of fault attacks. Section 3 presents our detailed analysis of the fault attack process and the post-reuse security of WOTS+. In Section 4, we describe our proposed fault-attacks resistant architecture, including WOTS+CS and SRCC. Finally, Section 5 details the experiments and evaluates the effectiveness of our countermeasures, followed by the conclusion.

2 Preliminaries

In this section, we first provide relevant background on the SPHINCS+ scheme and the fault attacks against SPHINCS+. SPHINCS+ is a stateless hash-based digital signature scheme that builds upon the SPHINCS scheme. It integrates several sub-algorithms, including FORS, WOTS+ [Mer89], and XMSS [BDH11]. SPHINCS+ employs tweakable hash functions and address (ADRS) to construct nodes, ensuring independence between nodes and positions while enhancing security. However, it remains vulnerable to fault attacks, which enable attackers to forge signatures with relative ease.

2.1 SPHINCS+ Scheme

Tweakable Hash Function. In the SPHINCS+, different hash functions are used depending on the operation. Table 1 summarizes all the hash functions and pseudorandom functions involved in SPHINCS+. Given a SPHINCS+ signing key $\mathrm{sk}_1, \mathrm{sk}_2$ and public key $\mathrm{pk}_1, \mathrm{pk}_2$, the "Parameters" column includes public and secret seeds that make hash function calls unique per key pair, and contextual information (i.e., ADRS, R, opt) that makes hash function calls unique per use. The set $\mathbf{B}$ denotes the set of bytes (i.e., $\mathbf{B} = \{0, 1, \ldots, 255\}$). The ADRS has a length of α bytes and n denotes the security parameter in bytes.

WOTS+. We abbreviate $\log_2$ as 'log'. Winternitz One-Time Signature (WOTS) segments a message of length n bytes into smaller parts, each consisting of $\log(w)$ bits. Let

$$l_1 = \left\lceil \frac{8n}{\log(w)} \right\rceil \ , \ l_2 = \left\lfloor \frac{\log(l_1 \cdot (w-1))}{\log(w)} \right\rfloor + 1 \text{ and } l = l_1 + l_2 \ . \tag{1}$$

Table 1: Tweakable Hash Function in SPHINCS+

Function	Parameters	Input	Output
$\mathbf{T}_l$	$(\mathrm{pk}_2, \mathrm{ADRS}) \in \mathbf{B}^n \times \mathbf{B}^\alpha$	$(x_1, \ldots, x_l) \in \mathbf{B}^{ln}$	$y \in \mathbf{B}^n$
$\mathbf{F}$	$(\mathrm{pk}_2, \mathrm{ADRS}) \in \mathbf{B}^n \times \mathbf{B}^\alpha$	$x \in \mathbf{B}^n$	$y \in \mathbf{B}^n$
$\mathbf{H}$	$(\mathrm{pk}_2, \mathrm{ADRS}) \in \mathbf{B}^n \times \mathbf{B}^\alpha$	$(x_1, x_2) \in \mathbf{B}^{2n}$	$y \in \mathbf{B}^n$
$\mathbf{PRF}$	$(\mathrm{ADRS}) \in \mathbf{B}^\alpha$	$\mathrm{sk}_2 \in \mathbf{B}^n$	$s \in \mathbf{B}^n$
$\mathbf{PRF_{msg}}$	$(\mathrm{sk}_1, \mathrm{opt}) \in \mathbf{B}^{2n}$	$msg \in \mathbf{B}^*$	$R \in \mathbf{B}^n$
$\mathbf{H_{msg}}$	$(\mathrm{pk}_2, \mathrm{pk}_1, R) \in \mathbf{B}^{3n}$	$msg \in \mathbf{B}^*$	$(md, idx) \in \mathbf{B}^m$

The message is divided into l_1 blocks, with a corresponding set of l_1 private keys. For each block, denoted its value by v where $v \in [w] = \{0, 1, \ldots, w-1\}$. In addition to the l_1 blocks representing the message, l_2 blocks are used to represent the checksum.

Interpret the message msg as a number and convert it into base w representation: $msg = v_0, \ldots, v_{l_1-1}$, where $v_i \in [w]$.

The WOTS+ scheme [ABB+22] is:

KeyGen(1^n):

1. Generate WOTS+ secret key from sk_2 and ADRS: $\mathrm{sk}^{\mathrm{W}} \leftarrow (s_0, \ldots, s_{l-1})$, where $s_i = \mathbf{PRF}(\mathrm{ADRS}, \mathrm{sk}_2)$ and sk_2 is the secret key seed in SPHINCS+.
2. Generate WOTS+ public key from sk^{W}: $\mathrm{pk}^{\mathrm{W}} = \mathbf{T}_l(\mathrm{pk}_2, \mathrm{ADRS}, p_0, \ldots, p_{l-1})$, $p_i = \mathbf{F}^{w-1}(\mathrm{pk}_2, \mathrm{ADRS}, s_i)$ and pk_2 is the public key in SPHINCS+.

Sign$(msg, \mathrm{sk}^{\mathrm{W}})$:

1. Segment the message msg into l_1 parts, and then interpret each part as a number, i.e., $\mathrm{Encode}(msg) = (v_0, \ldots, v_{l_1-1})$.
2. Calculate the checksum $cs = \sum_{i=0}^{l_1-1}(w-1-v_i)$. Segment the checksum cs into l_2 parts, and then interpret each part as a number, i.e., $\mathrm{Encode}(cs) = (v_{l_1}, \ldots, v_{l_1+l_2-1})$.
3. Calculate the WOTS+ signature: $\sigma^W \leftarrow (\sigma_0, \ldots, \sigma_{l-1})$ where $\sigma_i = \mathbf{F}^{v_i}(\mathrm{pk}_2, \mathrm{ADRS}, s_i)$.

Verify$(msg, \sigma^W, \mathrm{pk}^{\mathrm{W}})$:

1. Parse σ^W as $(\sigma_0, \ldots, \sigma_{l-1})$.
2. Calculate the checksum cs and encode message $(msg||cs)$ as signature location vector $\mathbf{v} = (v_0, \ldots, v_{l_1-1}, v_{l_1}, \ldots, v_{l-1})$.
3. For all $i \in [l]$, calculate $p_i = \mathbf{F}^{w-1-v_i}(\mathrm{pk}_2, \mathrm{ADRS}, \sigma_i)$.
4. Verify if $\mathrm{pk}^{\mathrm{W}} = \mathbf{T}_l(\mathrm{pk}_2, \mathrm{ADRS}, p_0, \ldots, p_{l-1})$ satisifies.

Merkle Tree and Authentication Path. In SPHINCS+, the foundational structure is the Merkle Tree. The value of each non-leaf node is obtained by concatenating its two children and applying the hash function $\mathbf{H}$, i.e., the value of the parent node is equal to $\mathbf{H}$(left child || right child). This process iteratively hashes pairs of nodes until a single root node remains, which is called "treehash".

The "authentication path" for a chosen leaf node refers to all sibling nodes adjacent to the nodes along the path from this leaf to the root node. For example, in Figure 1, the authentication path for the light-shaded leaf node consists of the three dark-shaded nodes.

FORS. In FORS, the message is divided into k blocks, each containing a bits. The entire FORS structure consists of k Merkle trees, each of them with height a and t leaf

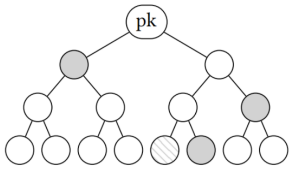

Figure 1: Authentication Path

nodes, where $t = 2^a$. For the i-th block of the message, its value m_i is used to select the m_i-th private key and its authentication path from the i-th Merkle tree as the i-th part of signature. The root nodes of the k trees are concatenated and then hashed to obtain the overall FORS public key.

XMSS and Hypertree. The overall structure of SPHINCS+ is that of a "hypertree." A SPHINCS+ hypertree is a complete $2^{h'}$-ary tree, where each node is an instance of XMSS, containing a Merkle tree of height h', referred to as a subtree.

- **Number of Layers** d and **Total Height** h: The hypertree's structure involves two parameters: the number of layers of subtrees, denoted by d, and the total height of the hypertree, denoted by h. For instance, a hypertree with parameters $h = 63, d = 7$ contains 7 layers of subtrees, where each subtree has a height of $63/7 = 9$ sub-layers.
- **Layer Ordering**: The layers are numbered from 0 to $d-1$, starting from the bottom layer and ascending towards the top.
- **Leaf Nodes of Subtrees**: The leaf node of the subtrees in the i-th layer is the public key of WOTS+, linked to it.

SPHINCS+. SPHINCS+ provides each signature block with its own private key pool. This enables the use of the same key pair for a larger number of signing operations. Additionally, SPHINCS+ introduces the concept of a private key seed, which, along with an address, serves as input to a pseudorandom function. SPHINCS+ utilizes a hypertree structure composed of multiple XMSS instances, with FORS integrated to the bottommost XMSS layer. WOTS+ is employed to connect different layers of the hypertree.

The SPHINCS+ signature SIG $= (R, \sigma^F, \sigma^{HT})$ is shown in Figure 6:

- R is a random number.
- σ^F is the signature of FORS.
- σ^W_λ is the signature of WOTS+, which connects the λ-th leaf node of the subtree.
- $auth_\lambda$ is the authentication path of the λ-th leaf node of the subtree.
- $\sigma^X_i = (\sigma^W_\lambda, auth_\lambda)$ is the signature of XMSS in the i-th layer.
- $\sigma^{HT} = (\sigma^X_0, \ldots, \sigma^X_{d-1})$ is the signature of Hypertree.

The procedures for key generation, signing, and verification in SPHINCS+ are described as follows:

Key generation. The SPHINCS+ signing key consists of two secret n-byte seeds sk_1 and sk_2 picked uniformly at random. sk_1 is used to choose a starting FORS in an unpredictable way. sk_2 is used to derive the key pairs of all the hash-based instances involved in the scheme. The SPHINCS+ public key consists of the public key pk_1 of hypertree as well as the public seed pk_2 that makes hash function calls unique per user.

Signing procedure. Given a SPHINCS+ signing key $(\mathrm{sk}_1, \mathrm{sk}_2)$, the scheme signs a message msg of arbitrary bit length with the following procedure:

1. Generate $R = \mathbf{PRF_{msg}}(\mathrm{sk}_1, \mathrm{opt})(msg)$ where $\mathrm{opt} \in \mathbf{B}^n$ is uniformly drawn at random to enable randomized signing.
2. Compute $(md, idx) = \mathbf{H_{msg}}(\mathrm{pk}_1, \mathrm{pk}_2, R)(msg)$.
3. Sign md with the FORS at idx using sk_2 to produce σ^F, and let pk^{F} be the FORS public key.
4. Sign pk^{F} with the hypertree to produce $\sigma^{HT} = (\sigma_0^X, \ldots, \sigma_{d-1}^X)$.

Verification procedure. Given a SPHINCS+ public key $(\mathrm{pk}_1, \mathrm{pk}_2)$, the scheme verifies that a SPHINCS+ signature $\mathrm{SIG} = (R, \sigma^F, \sigma^{HT})$ corresponds to the message msg with the following procedure:

1. Compute $(md, idx) = \mathbf{H_{msg}}(\mathrm{pk}_1, \mathrm{pk}_2, R)(msg)$.
2. Calculate pk^{F}, the public key of the FORS at idx, from md and σ^F.
3. Calculate $\mathrm{pk}^{\mathrm{HT}}$, the public key of the hypertree at the leaf index given by idx, from pk^{F} and $\sigma^{HT} = (\sigma_0^X, \ldots, \sigma_{d-1}^X)$.
4. Return true if $\mathrm{pk}^{\mathrm{HT}} = \mathrm{pk}_1$, false otherwise.

2.2 Fault Attacks and Countermeasures

Fault Attacks. The first fault attack against SPHINCS was proposed by Castelnovi et al. [CMP18]. In this attack, a fault is injected to force a WOTS+ key pair to sign an incorrect subtree root during the construction of any non-top-layer subtree. By obtaining another valid WOTS+ signature for the same subtree root, the attacker can easily forge a signature for this WOTS+ [GBH17]. Consequently, the portion of the SPHINCS signature linked below this WOTS+ is also forgeable. Combining the portion of the extracted SPHINCS signature linked above this WOTS+, the forged WOTS+ signature, and the forged SPHINCS signature linked below this WOTS+, the attacker can construct a complete forged SPHINCS signature. This process, similar to a tree grafting [Gen23], enables the forgery of an overall signature for any message.

However, SPHINCS+ enables randomized signing using a source of true randomness. This means for two SPHINCS+ signatures of the same message, the address of FORS will differ, making it harder for the attacker to find a WOTS+ to be reused. Due to the hypertree structure, the likelihood of WOTS+ reuse increases at higher layers. Consequently, the fault attack scenario becomes more complex in SPHINCS+.

Genêt [Gen23] extends the fault attacks from SPHINCS to SPHINCS+. The adversary executes universal forgery by using the grafted subtree and the value R' to forge a bottom part of the signature. The adversary then merges the extracted upper part with the forged part to craft a valid signature for the malicious message. The analysis in [Gen23] shows that a single unconstrained corruption of almost any computation has a catastrophic impact on the security guarantees of all SPHINCS+ parameter sets.

Countermeasures. Gavrilan et al. [GOW+24] further assume that the fault happens during the execution of hash functions, due to their cost compared to other instructions. To detect faults in the hash function and prevent the leakage of faulty signatures, Gavrilan et al. introduced an impeccable Keccak. Nevertheless, this approach demands 3.2 times the resources of the original implementation and can only detect errors of 3 bits or less.

Genêt [Gen23] proposes WOTS+ caching strategies aimed at preventing the re-computation of vulnerable WOTS+ signatures. WOTS+ caching statically stores all WOTS+ signatures and public keys for a fixed number of top layers (e.g., layers $d - c$ to $d - 1$). During signing, the device checks if the required WOTS+ instance belongs to a cached layer; if so, it retrieves the pre-computed value, effectively eliminating the

possibility of fault injection during computation for those layers. However, this approach incurs a substantial memory cost, making it impractical to cache more than a few top layers.

3 Fault Attacks Analysis

3.1 A Complete Fault Attack

We propose a comprehensive and practical fault attack methodology targeting SPHINCS+, with the entire process systematically divided into three distinct phases: **Fault Injection**, **Signature Acquisition** and **Signature Forgery**:

- **Phase 1: Fault Injection (Online)** - Causing a WOTS+ signature in SPHINCS+ to sign an incorrect root node with physical access to the signing device.
- **Phase 2: Signature Acquisition (Online)** - Obtaining another signature from the same WOTS+ instance by repeatedly querying the challenger (signer) for multiple signatures until one signature path contains the previously attacked WOTS+.
- **Phase 3: Signature Forgery (Offline)** - For any given message msg', we forge the lower portion of the corresponding SPHINCS+ signature offline. By concatenating this forged signature with the previously obtained upper portion, we construct a complete forged SPHINCS+ signature that successfully passes verification.

Algorithm 1: Fault Injection (Phase 1)

Require: Physical access to target SPHINCS+ signing device.
Ensure: $(\mathrm{SIG}_{(A)}, \sigma^W_{(A)}, \ell^*, \lambda)$ — a faulty WOTS+ signature and its address.

1: Induce a physical fault (e.g., clock glitch) during signing execution.
2: Obtain a SPHINCS+ signature $\mathrm{SIG}_{(A)}$ with faulty WOTS+ signature $\sigma^W_{(A)}$.
3: Parse the signature $\mathrm{SIG}_{(A)}$ to extract:
   - The WOTS+ signature $\sigma^W_{(A)}$,
   - Its layer index $\ell^* \in \{0, \ldots, d-1\}$,
   - Its leaf index (address) λ within layer ℓ^*.
4: **return** $(\mathrm{SIG}_{(A)}, \sigma^W_{(A)}, \ell^*, \lambda)$

Algorithm 2: Signature Acquisition (Phase 2)

Require: $(\mathrm{SIG}_{(A)}, \sigma^W_{(A)}, \ell^*, \lambda)$ from Algorithm 1.
Ensure: $(\mathrm{SIG}_{(B)}, \sigma^W_{(B)})$ — another WOTS+ signature at the same address.

1: **repeat**
2: Query the honest signer for a new SPHINCS+ signature on an arbitrary message.
3: Check if any WOTS+ instance in the path has address (ℓ^*, λ).
4: **until** such a signature $\mathrm{SIG}_{(B)}$ is found
5: Extract the corresponding valid WOTS+ signature $\sigma^{W(B)}$ from $\mathrm{SIG}^{(B)}$.
6: **return** $(\mathrm{SIG}_{(B)}, \sigma^W_{(B)})$

Algorithm 3: Signature Forgery (Phase 3)

Require: $(\mathrm{SIG}_{(A)}, \sigma^W_{(A)}, \mathrm{SIG}_{(B)}, \sigma^W_{(B)}, \ell^*, \lambda)$ from Algorithm 1 and Algorithm 2.
Require: Target message msg' to forge.

Ensure: A valid forged SPHINCS+ signature $\mathrm{SIG}_{\mathrm{forged}}$ for msg'.

Recover signature location vector $\mathbf{v}_{(A)}$ and $\mathbf{v}_{(B)}$ from $\sigma^W_{(A)}$ and $\sigma^W_{(B)}$.
Compute lower-bound vector $\mathbf{b} = \min(\mathbf{v}_{(A)}, \mathbf{v}_{(B)})$ component-wise.
repeat
    Sample a fresh random R'.
    Compute $(md', idx') = \mathrm{H}_{\mathrm{msg}}(\mathrm{pk}_1, \mathrm{pk}_2, R')(msg')$.
until idx' maps to the target WOTS+ at ℓ^*-th layer, i.e., $\lfloor idx/2^{(\ell^* \times h')} \rfloor = \lfloor idx'/2^{(\ell^* \times h')} \rfloor$.
Sign md' with idx' to obtain lower-layer signature: σ'^F, σ'^X_0, $\ldots$, $\sigma'^X_{\ell^*-1}$ using the attacker's key pair.
repeat
    Replace the last node (root child) with a random value in the authentication path of $\sigma'^X_{\ell^*-1}$.
    Compute the root at layer $\ell^* - 1$ using this new authentication path.
    Get the WOTS+ location vector $\mathbf{v}'$ from the root.
until $\mathbf{v}' \geq \mathbf{b}$.
Compute forged WOTS+ signature σ'^W of $\mathbf{v}'$ from $\sigma^W_{(A)}$ and $\sigma^W_{(B)}$.
Extract the $auth_\lambda$ of ℓ^*-th layer from $\mathrm{SIG}_{(A)}$ or $\mathrm{SIG}_{(B)}$, get $\sigma'^X_{\ell^*} = (\sigma'^W, auth_\lambda)$.
Construct the full forged SPHINCS+ signature:

$$\mathrm{SIG}_{\mathrm{forged}} = (\underbrace{R'}_{\text{sought}}, \underbrace{\sigma'^F, \sigma'^X_0, \ldots, \sigma'^X_{\ell^*-1}}_{\text{forged}}, \underbrace{\sigma'^X_{\ell^*}}_{\text{grafted}}, \underbrace{\sigma^X_{\ell^*+1}, \ldots, \sigma^X_{d-1}}_{\text{extracted}})$$

where the upper-layer $\sigma^X_{\ell^*+1}$ to σ^X_{d-1} are copied from either $\mathrm{SIG}_{(A)}$ or $\mathrm{SIG}_{(B)}$.
return $\mathrm{SIG}_{\mathrm{forged}}$

3.2 Post-Reuse WOTS+ Security

In this section, we present a formal analysis of the security degradation of WOTS+ after a key is **reused twice**. Previous studies have attempted to quantify this risk: Groot Bruinderink and Hülsing [GBH17] provided an approximate formula based on simplified independence assumptions, while Fluhrer [Flu23] leveraged Monte Carlo simulations combined with dynamic programming (DP) to compute the WOTS+ post-reuse security. This yielded a more accurate average for post-reuse security and demonstrated discrepancies in [GBH17]'s formula. However, the DP method is computationally intensive. To address this issue, we introduce a method based on **Generating Functions (GFs)**. This approach not only computes the exact forgery probability but does so with higher efficiency, enabling large-scale simulations.

3.2.1 Formalization of the Two-Message Attack

Assume the output of the hash function is independently and uniformly distributed. Let w be the Winternitz parameter. A WOTS+ signature location vector $\mathbf{v}$ is defined as

$$\mathbf{v} = \mathbf{m} \,\|\, \mathbf{c} = (v_0, v_1, \ldots, v_{l-1}) \in \{0, 1, \ldots, w-1\}^l, \tag{2}$$

where $\mathbf{m} = (m_0, m_1, \ldots, m_{l_1-1})$ denotes the encoded message blocks, $\mathbf{c} = (c_0, c_1, \ldots, c_{l_2-1})$ denotes the encoded checksum blocks, and $l = l_1 + l_2$.

When a WOTS+ key pair is reused to sign two independent, uniformly random messages, the attacker observes the corresponding signatures. Due to the forward-only nature of hash chains, from a signature node at position v_i, the attacker can compute signature nodes for any position $v'_i \geq v_i$. This capability is captured by the following definitions.

Definition 1 (Attacker Capability)**.** Given two signature location vectors $\mathbf{v}_{(A)}$ and $\mathbf{v}_{(B)}$ from a key reuse, the attacker's knowledge is summarized by the **lower bound vector** $\mathbf{b} = (b_0, \ldots, b_{l-1}) \in \{0, \ldots, w-1\}^l$, defined as:

$$b_i = \min(v_{(A)i}, v_{(B)i}), \quad \text{for } 0 \leq i < l. \tag{3}$$

Definition 2 (Instance-Specific Forgery Probability)**.** For a fixed lower bound vector $\mathbf{b}$, let $\mathbf{v}' = \mathbf{m}' \| \mathbf{c}'$ be the location vector of a randomly chosen target message $\mathbf{m}'$. The attacker succeeds if $v'_i \geq b_i$ for all i. The forgery probability for this instance is

$$P_s(\mathbf{b}) = \frac{N_{\text{forge}}(\mathbf{b})}{w^{l_1}}, \tag{4}$$

where $N_{\text{forge}}(\mathbf{b})$ is the number of messages $\mathbf{m}'$ whose location vectors satisfy $v'_i \geq b_i$ for all i.

Definition 3 (Average Security Level)**.** Let $\mathbf{B}$ denote the random lower bound vector induced by two independent random signatures. The average forgery probability is

$$P_{\text{avg}} = \mathbb{E}[P_s(\mathbf{B})] = \sum_{\mathbf{b}} P_s(\mathbf{b}) \cdot \Pr[\mathbf{B} = \mathbf{b}], \tag{5}$$

where $\Pr[\mathbf{B} = \mathbf{b}]$ is the probability of obtaining $\mathbf{b}$ by randomly sampling $\mathbf{v}_{(A)}$ and $\mathbf{v}_{(B)}$. The **average security level** is then defined as $-\log_2(P_{\text{avg}})$ bits.

3.2.2 Exact Computation of P_s via Generating Functions

To compute the instance-specific forgery probability $P_s(\mathbf{b})$, we must determine $N_{\text{forge}}(\mathbf{b})$, the number of valid messages $\mathbf{m}'$ whose location vectors satisfy the lower bound $v'_i \geq b_i$ for all i. We propose a hybrid algorithm that combines generating functions with explicit enumeration to compute this value efficiently.

First, we formalize the relationship between the increment values of the message and the checksum.

Lemma 1 (Checksum Linear Constraint)**.** *Let $\mathbf{b} \in \{0, \ldots, w-1\}^l$ be the lower bound vector derived from two observed signatures. A forged signature $\mathbf{v}' = \mathbf{b} + \boldsymbol{\delta}$ is valid if and only if the increments $\boldsymbol{\delta} = (\delta_0, \ldots, \delta_{l-1})$ satisfy:*

$$0 \leq \delta_i \leq w - 1 - b_i \quad \textit{for all } i, \tag{6}$$

and satisfy the WOTS+ checksum equality:

$$\sum_{i=0}^{l_1-1} \delta_i + \sum_{j=0}^{l_2-1} w^j \cdot \delta_{l_1+j} = \Delta(\mathbf{b}), \tag{7}$$

where $\Delta(\mathbf{b})$ is the total "slack" available, defined as:

$$\Delta(\mathbf{b}) = l_1(w-1) - \sum_{i=0}^{l_1-1} b_i - \sum_{j=0}^{l_2-1} w^j \cdot b_{l_1+j}. \tag{8}$$

Proof. The WOTS+ checksum condition requires $\sum_{i=0}^{l_1-1}(w-1-v'_i) = \sum_{j=0}^{l_2-1} v'_{l_1+j} \cdot w^j$. Substituting $v'_k = b_k + \delta_k$ and separating the variables δ from the constants b yields Equation (7). □

Based on Lemma 1, the counting problem reduces to finding the number of integer solutions to Equation (7) subject to the bounds in Equation (6). Since l_2 (the number of checksum blocks) is small (typically 2 or 3), while l_1 is large (e.g., 67), we employ a generating function for the message part and iterate over the checksum part.

Theorem 1 (Hybrid Counting Formula)**.** *Let $\Phi_{\mathbf{b}}(z)$ be the generating function for the valid increments of the message blocks:*

$$\Phi_{\mathbf{b}}(z) = \prod_{i=0}^{l_1-1} \left(\sum_{k=0}^{w-1-b_i} z^k \right). \tag{9}$$

The total number of valid forgeries $N_{\text{forge}}(\mathbf{b})$ is given by summing the coefficients of $\Phi_{\mathbf{b}}(z)$ corresponding to all valid checksum configurations. Here, $[z^d]\Phi_{\mathbf{b}}(z)$ denotes the coefficient of z^d in the polynomial $\Phi_{\mathbf{b}}(z)$:

$$N_{\text{forge}}(\mathbf{b}) = \sum_{\delta_{l_1},\ldots,\delta_{l-1}} \left[z^{R(\boldsymbol{\delta}_{chk})} \right] \Phi_{\mathbf{b}}(z), \tag{10}$$

where the summation runs over all valid checksum increments $\delta_{l_1+j} \in [0, w-1-b_{l_1+j}]$, and the target degree $R(\boldsymbol{\delta}_{chk})$ is determined by the remaining slack from Lemma 1:

$$R(\boldsymbol{\delta}_{chk}) = \Delta(\mathbf{b}) - \sum_{j=0}^{l_2-1} w^j \cdot \delta_{l_1+j}, \text{ with } \boldsymbol{\delta}_{chk} = (\delta_{l_1}, \ldots, \delta_{l-1}). \tag{11}$$

If $R(\boldsymbol{\delta}_{chk}) < 0$, the coefficient is 0.

Proof. Equation (7) from Lemma 1 can be rewritten as $\sum_{i=0}^{l_1-1} \delta_i = \Delta(\mathbf{b}) - \sum_{j=0}^{l_2-1} w^j \delta_{l_1+j}$. The generating function $\Phi_{\mathbf{b}}(z)$ enumerates all possible sums of the message increments $\sum_{i=0}^{l_1-1} \delta_i$, where the coefficient of z^k represents the number of ways to achieve a sum of k with the constraint (6). By fixing a valid checksum increment vector $\boldsymbol{\delta}_{chk}$, the required sum from the message part becomes fixed at $R(\boldsymbol{\delta}_{chk})$. Summing the valid counts for all possible $\boldsymbol{\delta}_{chk}$ yields the total number of forgeries. □

Implementation Note: Directly expanding $\Phi_{\mathbf{b}}(z)$ involves polynomial multiplication. In our SageMath implementation (successful_forgery_probability_fast), we optimize this by observing that $\sum_{k=0}^{U} z^k = \frac{1-z^{U+1}}{1-z}$. Thus, $\Phi_{\mathbf{b}}(z) = G(z) \cdot (1-z)^{-l_1}$, where $G(z) = \prod_{i=0}^{l_1-1}(1-z^{w-b_i})$. We compute the coefficients of $G(z)$ explicitly and then convolve them with the binomial coefficients of the Taylor series expansion of $(1-z)^{-l_1}$: $(1-z)^{-l_1} = \sum_{k=0}^{+\infty} \binom{k+l_1-1}{l_1-1} z^k$, accelerating the computation compared to dynamic programming.

Simulation Strategy: Calculating the exact average P_{avg} via full enumeration is computationally infeasible due to the vast space of signature pairs. Instead, we employ a Monte Carlo method:

1. Sample $N = 100,000$ pairs of $(\mathbf{v}_{(A)}, \mathbf{v}_{(B)})$.
2. For each pair, derive $\mathbf{b}$ and compute $P_s(\mathbf{b})$ using Theorem 1.
3. Estimate the distribution of security levels $-\log_2(P_s)$ and the average security.

3.3 Simulation Results and Analysis

Using the methodology from Subsection 3.2, we executed Monte Carlo simulations for 100,000 reuse instances across all parameter sets. The simulated result is shown in Figure 2

Table 2: WOTS+ Post-Reuse Security Levels P_{avg} for Various Parameters: Comparison of Simulated Results and Approximate Estimates.

Parameters	Simulated Result (bits)	Approximate Estimate [GBH17] (bits)
$w = 16, n = 128$	$P_{avg} = 18.4$	$P_{avg} = 18.2$
$w = 16, n = 192$	$P_{avg} = 35.6$	$P_{avg} = 26.5$
$w = 16, n = 256$	$P_{avg} = 35.4$	$P_{avg} = 34.9$

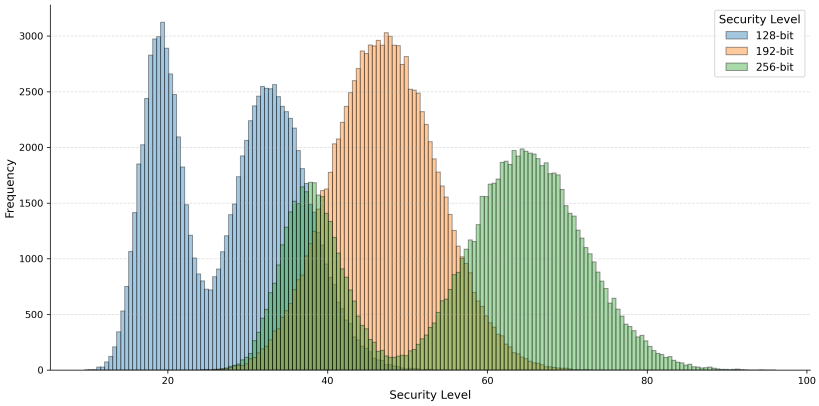

Figure 2: The probability distribution of the instance-specific security levels, defined as $-\log_2(P_s(\mathbf{b}))$, derived from 100,000 Monte Carlo samples.

and the average security levels (P_{avg}) are summarized in Table 2, compared with previous approximate estimates [GBH17].

The approximate estimates from [GBH17] are shown to have considerable error for 192-bit security. It is also noteworthy that the 256-bit post-reuse security is lower than the 192-bit security. This is due to the bimodal distribution of post-reuse security probabilities observed in both 128-bit and 256-bit security levels in Figure 2.

Most Significant block Value (MSV). In the 128-bit and 256-bit cases, the most significant block value (MSV) of the checksum can take on 2 different values, whereas for 192-bit security, the checksum's MSV is almost exclusively a single value, as shown in Table 3. For instance, consider the 128-bit case: the checksum's value ranges from 0 to 480, with a mean of 240, approximating a normal distribution. When the checksum falls between 0 and 255, the MSV is 0. If the checksum's value is between 256 and 480, the MSV becomes 1. Fluhrer [Flu23] first observed that having two possible MSV values in 128-bit and 256-bit WOTS+ reduces post-reuse security. In this work, we give a deep analysis of the underlying reasons for this phenomenon and propose a specific solution.

Table 3: Distribution of Checksum Most Significant Value for WOTS+ Parameters

Parameters	MSV=0	MSV=1	MSV=2	same	diff
$w = 16, n = 128$	72.3%	27.7%	0%	59.9%	40.1%
$w = 16, n = 192$	0%	100%	0%	100%	0%
$w = 16, n = 256$	0%	80.3%	19.7%	68.4%	31.6%

For example, for 128-bit WOTS+, a randomly chosen message has a 72.3% probability that its checksum's MSV is 0, and a 27.7% probability that it is 1. This means that for

two randomly given independent messages, there is a 40.1% chance that the MSVs of their checksums differ (i.e., one is 1 and the other is 0). Conversely, there is a 59.9% chance that they are the same.

When a message is signed with WOTS+, it is divided into l_1 blocks. Each block corresponds to a digit with a value ranging from $[0, w-1]$. When the digits of each block are generally small, the corresponding checksum value tends to be larger, which typically results in its MSV being 1. Conversely, when the digits of each block are generally large, the corresponding checksum value tends to be smaller, leading to its MSV being 0. A smaller digit for a specific block corresponds to the signature node being in a lower position within its WOTS+ chain. Conversely, a larger digit means the node is in a higher position. Given a node value at a lower position in a WOTS+ chain, all node values above it can be computed.

Impact of MSV Combinations on Security. Let us now consider three distinct scenarios based on the MSV of the checksums from two independent messages:

- Scenario (0,0): Two independent messages with checksum MSVs of (0, 0): Both messages have generally large block digits, positioning their signature nodes high in WOTS+ chains. This inherently limits the number of forgeable messages.
- Scenario (1,1): Two independent messages with checksum MSVs of (1, 1): Both messages feature generally small block digits, placing signature nodes low in WOTS+ chains. This condition usually allows for more potential forgeries. If a forged message's block digits are generally large, its checksum's MSV would become 0. However, the initial two messages did not provide a node value corresponding to this '0' state. Therefore, the ability to forge is restricted to messages with generally small block digits, making it impossible to forge messages where block digits are generally large.
- Scenario (0,1): Two independent messages with checksum MSVs of (0, 1): One message has generally small block digits (low chain position), while the other has large block digits (high chain position). This combination expands the forging capability, effectively encompassing properties of both (0,0) and (1,1) scenarios. This leads to a substantially higher number of forgeable messages and, consequently, a lower security level.

This also explains the bimodal distribution of post-reuse security probabilities observed in both 128-bit and 256-bit security levels in Figure 2. The left peak (corresponding to lower post-reuse security) indicates instances where the checksum MSVs of the first two message hashes differ. The right peak (corresponding to higher post-reuse security) represents cases where the checksum MSVs of the first two message hashes are the same.

Experimental Validation and Statistical Analysis. In our 128-bit sampling experiment, we observed the following probabilities for different combinations of the two messages' most significant values corresponding to specific security levels. Here, a post-reuse security level below 25 bits falls into the left peak of the distribution, while values above 25 bits fall into the right peak. As illustrated in Table 4, the left peak primarily corresponds to the (1, 0) scenario. Conversely, the right peak predominantly comprises the (0, 0) and (1, 1) scenarios.

When the checksum's MSV is restricted to a single value, we do not observe a bimodal distribution (i.e., a significant probability of lower security). In such a scenario, the second most significant value effectively becomes the dominant factor, similar to how the MSV typically behaves. However, unlike the MSV, this second most significant value is not limited to one or two states; it can take on w different values.

Taking the 192-bit case as an example, the probabilities of the second MSV in the 192-bit security level are shown in Figure 3. We might look for extreme cases like the two

Table 4: Probability Distribution of Post-Reuse Security Levels Based on Checksum Most Significant Value Combinations (128-bit WOTS+)

Post-Reuse Security	Scenario (0,0)	Scenario (1,1)	Scenario (0,1)
Low (< 25 bits)	0.9%	0.4%	**38.5%**
High (> 25 bits)	**52.2%**	7.4%	0.6%

messages having second most significant values of 0 and $w-1$ considering lower security. This mirrors Scenario (0,1) described previously (where MSVs are different). However, the extreme combinations such as $(0, w-1), (1, w-1), (0, w-2)$ occur rarely, though they lead to lower security. The "extreme" cases under Scenario (0,1) are diluted by the broader range of possible most significant values. Therefore, the bimodal distribution only emerges when the checksum's most significant value can take on two distinct possibilities.

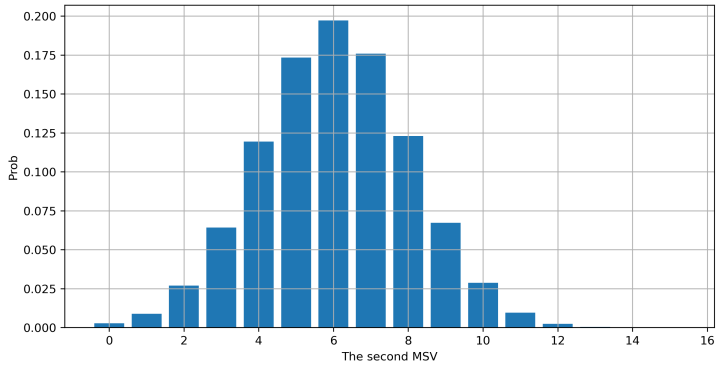

Figure 3: Probability Distributions of the second MSV for w=16 and n =192.

4 Fault Attack-Resistant Architecture

4.1 WOTS+CS: WOTS+ with Checksum Segmentation

In our analysis in Subsection 3.3, we observed that the post-reuse security probability distribution in WOTS+ exhibits a bimodal (two-peak) shape when the MSV of the checksum has only two possible values. This bimodal distribution leads to lower average post-reuse security compared to scenarios where the MSV has a single value. For instance, the average post-reuse security for 256-bit WOTS+ (bimodal) is lower than that for 192-bit WOTS+ (unimodal).

Based on this finding, we propose an improved version of WOTS+, called WOTS+ with Checksum Segmentation (WOTS+CS), designed to enhance the post-reuse security of WOTS+ in 128-bit and 256-bit instantiations. The core idea behind WOTS+CS is to modify the number of message blocks covered by the checksum. This modification ensures that the checksum's MSV either has only one possible value or more than two values, thereby preventing the bimodal post-reuse security probability distribution.

Let us consider the 128-bit WOTS+ as an example. Here, the checksum's value ranges from 0 to 480. Since $480 > 16^2$, three blocks are required to encode the checksum's value. In this case, the MSV is 1 when the checksum is greater than 255, and 0 otherwise. This results in only two possible MSV values, leading to the bimodal distribution.

In contrast, for a hypothetical 64-bit WOTS+ (assuming the message length is reduced from 128-bit to 64-bit), the checksum's value ranges from 0 to 240. Since $240 < 16^2$, only

two blocks are needed to encode the checksum. This allows the checksum's MSV to take on values from [0, 15], similar to the 192-bit case, as illustrated in Figure 3. Consequently, the post-reuse security probability distribution in this scenario does not exhibit a bimodal shape.

Since message digest lengths (128, 192, 256 bit) in WOTS+ are all divisible by 64, WOTS+CS leverages this property. In WOTS+CS, the message blocks are divided into multiple 64-bit segments, with each segment having its own checksum. For example, in 128-bit WOTS+CS, the message blocks are split into two parts, each with its own checksum. Each of these checksums is encoded using two blocks, resulting in a total of four blocks for a complete 128-bit WOTS+CS checksum. In essence, a 128-bit WOTS+CS can be viewed as the combination of two independent 64-bit WOTS+ instances. As a single 64-bit WOTS+ does not exhibit a bimodal distribution, the combination of two such instances will also result in a unimodal distribution, as depicted in Figure 4.

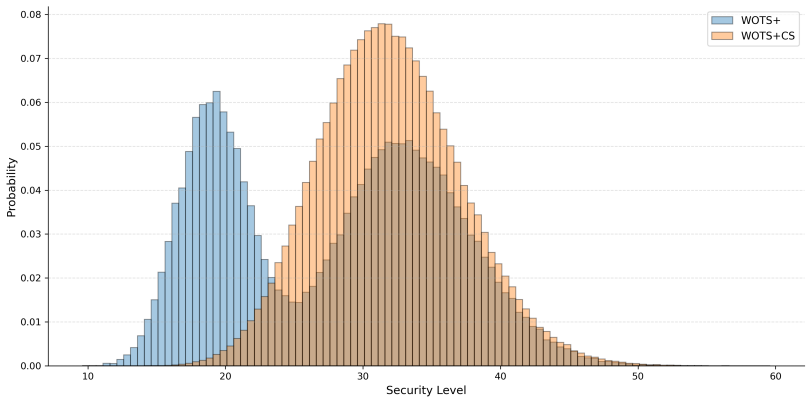

Figure 4: Comparing Post-Reuse Security Probability Distributions: WOTS+ vs. WOTS+CS (128-bit)

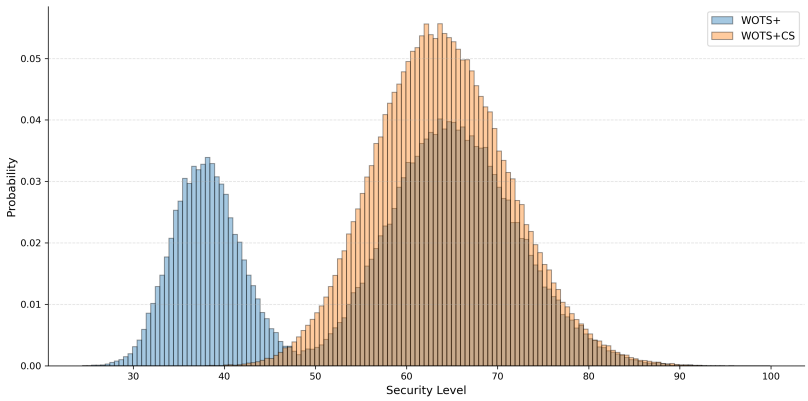

Figure 5: Comparing Post-Reuse Security Probability Distributions: WOTS+ vs. WOTS+CS (256-bit)

The same principle applies to 256-bit WOTS+CS. Here, the message blocks are divided into four parts, resulting in a total of eight blocks for a complete 256-bit WOTS+CS checksum. Figure 5 illustrates the probability distribution of post-reuse security levels for WOTS+CS compared to WOTS+ in the 256-bit setting, demonstrating that WOTS+CS

successfully avoids the bimodal distribution.

Table 5 presents a comparison of the average post-reuse security levels for WOTS+CS and WOTS+. As the table shows, WOTS+CS improves the average post-reuse security for both 128-bit and 256-bit WOTS+ instantiations, at the cost of a small increase in the total number of blocks. There is also a slight improvement for 192-bit, which we hypothesize, is due to a more efficient utilization of the checksum blocks.

Table 5: Average post-reuse Security and Total Block Count for WOTS+ and WOTS+CS

	Average post-reuse Security		Total Number of Blocks	
	WOTS+	WOTS+CS	WOTS+	WOTS+CS
$w = 16, n = 128$	18.4	24.9	35	36
$w = 16, n = 192$	35.6	37.3	51	54
$w = 16, n = 256$	35.4	49.8	67	72

4.2 Subtree Root Children Caching

While existing countermeasures like recalculations [ALCZ20], impeccable Keccak circuits [GOW+24], and WOTS+ caching aim to mitigate these, they incur overheads in terms of computation time or hardware resources respectively.

We propose **Subtree Root Children Caching (SRCC)**, which does not require additional computation time or substantial hardware resources, and needs only a small amount of storage space. The core idea of fault attacks is to cause a WOTS+ to sign two or more messages (which, in SPHINCS+, are essentially the subtree roots). The most straightforward approach is to prevent WOTS+ from signing different subtree roots.

Therefore, if we store the subtree roots during the key generation and compare the computed root with the stored one during signing, then a mismatch would indicate a fault attack, which requires re-computation.

Our strategy extends beyond merely caching subtree roots; it also stores their two children and the corresponding WOTS+ public key (i.e., leaf nodes of the next upper layers). This caching of WOTS+ public keys eliminates redundant computations during signature generation, thereby substantially enhancing SPHINCS+ signing speed. Specifically, during authentication path calculations, utilizing one of the cached child nodes directly allows us to save half of the hash computations within that subtree. Compared to the method of [Gen23] by caching the WOTS+ signatures and their public keys, our method reduces the storage memory and has a better performance.

However, storing all the subtree roots, their children and leaf nodes require an amount of storage space. Due to the structure of the SPHINCS+ hypertree and the variability of idx, WOTS+ signatures in the lower layers are more difficult to attack, and even if an attack occurs, it is difficult for the attacker to forge a signature. Conversely, higher layer WOTS+ signatures are more vulnerable to attacks, and forging a signature is also easier. Therefore we choose to store all tuples of four nodes containing the subtree root on the top few layers. The specific nodes stored in our scheme are illustrated in Figure 6. Overall, the SRCC algorithm is shown as follows:

Algorithm 4: SRCC Signing Procedure for XMSS in Caching Layers

Require:

c: number of cached layers ($1 \leq c < d$)
d: total number of hypertree layers
ADRS_j: the address of this XMSS at layer j
Cached data for layers $j \in \{d-c, \ldots, d-1\}$:
RT_{j-1}: subtree root from layer $j-1$

pk_j^W: cached WOTS+ public key linking RT_{j-1}
$\mathrm{RCA}_j, \mathrm{RCC}_j$: cached left/right children of the subtree root at layer j
Ensure: Signature $\sigma^X = (\sigma_j^W, \mathrm{auth}_j)$ for layer j
if $j = d - c - 1$ **then** ▷ Bottom cached layer
  Read $\mathrm{RCA}_j, \mathrm{RCC}_j$ from cache (with $\mathrm{RCA}_j \in \mathrm{auth}_j$)
  repeat
    $\sigma_j^W, \widehat{\mathrm{pk}}_j^W \leftarrow \mathrm{WOTS_Sign}(\mathrm{RT}_{j-1})$
    $\mathrm{auth}_j \setminus \{\mathrm{RCA}_j\} \leftarrow \mathrm{Auth_Sign}(\mathrm{ADRS}_j)$
    $\widetilde{\mathrm{RCC}}_j \leftarrow \mathrm{Auth_Sign}(\widehat{\mathrm{pk}}_j^W, \mathrm{auth}_j \setminus \{\mathrm{RCA}_j\})$
  until $\widetilde{\mathrm{RCC}}_j = \mathrm{RCC}_j$
  return $(\sigma_j^W, \mathrm{auth}_j)$
else if $d - c \leq j \leq d - 2$ **then** ▷ Middle cached layers
  repeat
    Read RT_{j-1} and cached pk_j^W
    $\sigma_j^W, \widehat{\mathrm{pk}}_j^W \leftarrow \mathrm{WOTS+Sign}(\mathrm{RT}_{j-1})$
    if $\widehat{\mathrm{pk}}_j^W \neq \mathrm{pk}_j^W$ **then**
      continue ▷ Fault detected: retry signing
    end if
    Read $\mathrm{RCA}_j, \mathrm{RCC}_j$ and half of cached WOTS+ public keys at layer j from cache
    $\mathrm{auth}_j \setminus \{\mathrm{RCA}_j\} \leftarrow \mathrm{Auth_Sign}$(half of cached WOTS+ public keys)
    $\widetilde{\mathrm{RCC}}_j \leftarrow \mathrm{Auth_Sign}(\widehat{\mathrm{pk}}_j^W, \mathrm{auth}_j \setminus \{\mathrm{RCA}_j\})$
  until $\widetilde{\mathrm{RCC}}_j = \mathrm{RCC}_j$
  return $(\sigma_j^W, \mathrm{auth}_j)$
else if $j = d - 1$ **then** ▷ Top layer
  repeat
    Read RT_{j-1} and cached pk_j^W
    $\sigma_j^W, \widehat{\mathrm{pk}}_j^W \leftarrow \mathrm{WOTS+Sign}(\mathrm{RT}_{j-1})$
  until $\widehat{\mathrm{pk}}_j^W = \mathrm{pk}_j^W$
  Read all cached WOTS+ public keys at layer j from cache
  $\mathrm{auth}_j \leftarrow \mathrm{Auth_Sign}$(all cached WOTS+ public keys)
  return $(\sigma_j^W, \mathrm{auth}_j)$
end if

It is worth noting that caching $(\mathrm{pk}^W, \mathrm{RCA}, \mathrm{RCC})$ serves solely to accelerate the signing process. The fault resilience is derived exclusively from the stored subtree root (RT). Therefore, in memory-constrained devices, one can choose to store only the subtree roots to achieve the exact same security guarantees.

Experimental results indicate that for cached layers, the fault injection attempts are ineffective. So the adversary can only target the uncached layers.

Let 2^S (KB) represent the maximal caching size of the SRCC, and $d-1-c$ represent the lowest fully cached layer. As a result, the condition $(2^S \times 2^{10})/n \geq 4(2^{h'} + 2^{h' \cdot 2} + \cdots + 2^{h' \cdot c})$ must be satisfied, and the value of c is calculated as:

$$c = \left\lfloor \frac{1}{h'} \log_2 \left(\frac{2^S \times 2^8}{n} (1 - 2^{-h'}) + 1 \right) \right\rfloor$$

In the 128-bit small variant, $h' = 9$. In case of $c = 1$, to store all subtree roots, their children and corresponding public keys of WOTS+ of the layer $d - 1$, the total storage required is $4 \times 2^{h'} \times$ (node size). Each node is $16 = 2^4$ bytes, then the total storage demand is $4 \times 2^9 \times 2^4 = 32$K bytes. Since the root nodes of the cached layer are stored, attackers

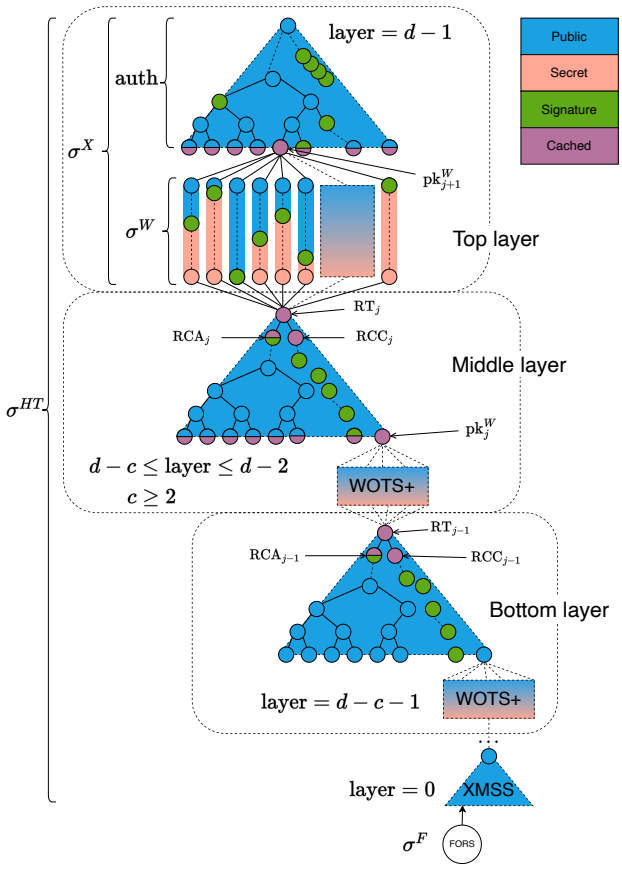

Figure 6: SPHINCS+ Signature with SRCC

are forced to target WOTS+ instances in the next layer down. This subsequent layer contains $2^{9\times 2} = 2^{18}$ root nodes. Consequently, in Phase 2 of the attack, the adversary must query an expected 2^{18} signatures to obtain another signature for the specific WOTS+ instance they have already faulted. This drastically increases the effort required for a successful attack.

Comparatively, to fully store the same number of layers, [Gen23] (caching WOTS+ signatures and public keys) requires a caching size 9 times larger in 128-bit security level, 13 times larger in 192-bit security level and 17 times larger in 256-bit security level to achieve the same effect.

For the performance, SRCC algorithm reduces the hash function calls in the signing procedure by $2^{h'-1}(lw+1)+2^{h'-1}$ times for the bottom layer; by $(2^{h'}-1)(lw+1)+2^{h'-1}$ times each layer for total $c-1$ middle layers; by $(2^{h'}-1)(lw+1)-(\frac{1}{2}lw+1)$ times on average for the top layer. In total, SRCC algorithm decreases the signing procedure complexity by

$$\left(c(2^{h'}-1)+2^{h'-1}-\frac{1}{2}\right)(lw+1)+c2^{h'-1}-\frac{1}{2}$$

hash function calls on average. Compared to the caching countermeasure of [Gen23], which decreases the signing procedure complexity by $c2^{h'}(lw+1)$ hash function calls, our method provides a performance advantage in signature generation.

Another difference between the caching method of [Gen23] and ours is that it cannot detect the fault attack. By the method of [Gen23], even if the subtree root is corrupted, the signer still uses the cached WOTS+ signature to sign the incorrect subtree root. This

leads to a faulty SPHINCS+ signature that cannot be verified. However, the signer is not aware of it. Instead, our method can detect the fault injected to the cached layers and automatically correct it, which is more secure.

It is worth noting the impact of our countermeasures on Known Answer Test (KAT) compatibility. Since SRCC is strictly an implementation-level optimization that caches intermediate values without altering the cryptographic outputs, it maintains full KAT compatibility with standard SPHINCS+. Conversely, because WOTS+CS structurally modifies the checksum algorithm to enhance security, it represents an algorithmic change and would thus require updated KATs for standardization.

5 Experiment

5.1 Fault Injection Experiment

Subtree root children caching (SRCC) can mitigate Phase 1 (fault injection), though it requires a small amount of additional storage. WOTS+CS, on the other hand, provides resilience against Phase 3 (signature forgery) of fault attacks, at the cost of a slight increase in signature length.

It is impractical to implement SRCC across all subtrees in SPHINCS+ (approximately 2^{65} subtrees), meaning it can only be applied to some layers. Given that higher layers in a hypertree contain fewer subtrees, while lower layers have more, an attacker conducting a Phase 1 fault injection attack would ideally target a high-layer WOTS+ instance to obtain a faulty signature. This strategy increases the probability of encountering the same subtree when acquiring another WOTS+ signature in Phase 2, reducing the number of attempts needed. Consequently, WOTS+ instances in higher layers of SPHINCS+ are more susceptible to attack. Based on this, we propose implementing SRCC in the higher-layer subtrees.

For the lower-layer subtrees, we suggest using WOTS+CS instead of WOTS+. Attackers already need to query more signatures in Phase 2 to encounter the same subtree at these lower layers. The additional protection provided by WOTS+CS in Phase 3 further compounds the difficulty for attackers to forge signatures.

Hardware. As the fault attacks do not require sophisticated glitching technology, our proof of concept uses the ChipWhisperer framework to perform experiments, which includes:

- The ChipWhisperer-Lite Level 1 starter kit.
- A NAE-CW308T-STM32F3 as the Device Under Testing (DUT).
- A desktop computer running Ubuntu 22.04.5 LTS.

Software. We modified the attack implementation of SPHINCS+ from [Gen23]. Our enhanced implementation now outputs the subtree root alongside the signature result. This addition is crucial because it accounts for scenarios where a signature might be incorrect, yet its corresponding subtree root node remains valid. The instance attacked is sphincs-shake-128f-simple. The hash function SHAKE was instantiated with a portable software implementation. We assume a fault occurs during a hash function call, causing its output to completely deviate from the intended value.

For practical purposes, the software in DUT was further modified to limit the signing procedure to the computation of a single layer. As a result, the software would use the WOTS+ key pair of an XMSS at a fixed layer $0 < l^* < d$ to sign the subtree root at layer $l^* - 1$ addressed by a given index. The output signature consists of the WOTS+ signature along with the authentication path in the XMSS of layer l^*. The desktop PC handles the remaining parts of SPHINCS+ including FORS.

The desktop PC communicates with DUT through UART and the protocol is implemented using ChipWhisperer's SimpleSerial library. The DUT can be commanded to:

- Program the SPHINCS+ secret and public seeds sk_1 and pk_2.
- Given an address, compute the WOTS+ signature and the authentication path of the XMSS at layer l^*.
- Retrieve the bytes of the last WOTS+ signature and the authentication path computed.

Fault injection. To collect faulty signatures, the ChipWhisperer is used to inject a glitch in the system clock of the DUT. We do not synchronize the glitch injection with a trigger signal as we do not require hitting a precise instruction to collect exploitable signatures. Instead, the glitch is manually injected after a (progressive) delay that follows the communication with the DUT.

In the experiment, we simulate the layer caching countermeasure (see Subsection 4.2) by pretending that SRCC is used in layer $d-2$. In practice, such a storage would amount to only 32 KB of ROM. The experiment therefore aims to show the feasibility of an attack on the third-to-top layer (i.e., $l^* = d-3$).

The DUT takes around 28 seconds to compute a single XMSS authentication path and WOTS+ signature, during which the clock glitch is blindly injected. We conduct N = 5 trials where a single trial consists of repeating the above procedure with a fixed SPHINCS+ secret seed to collect 1,024 potentially faulty signatures. The glitch characteristics were explored experimentally to favor faulty signatures. We set the external offset to 37 samples, use a width of 7 samples and a clock offset of 5 samples.

To simulate an unprofiled 'blind' attack, we injected glitches at uniformly distributed random times during signing. It is worth noting that targeting particular operations, such as hashing to merge the two root children into the subtree root or hashing a node with its sibling in the authentication path, would indeed increase the fault injection success rate.

Table 6: Analysis of the collected signatures in N = 5 trials of fault attacks against sphincs+-shake-128f-simple

Experiment	Correct Signature	Faulty Signature	Faulty Root	Faulty Signature Root Correct
Exp 1	844	180	172	8
Exp 2	860	164	156	8
Exp 3	839	185	176	9
Exp 4	849	175	164	11
Exp 5	850	174	162	12

Table 6 presents the outcomes of five distinct fault injection experiments (Exp 1 through Exp 5) conducted on the signature generation process. Each experiment quantifies specific fault occurrences observed during signature creation. "Correct Signature" denotes the number of successfully generated and valid signatures. "Faulty Signature" represents the total number of times an incorrect or invalid signature was produced. "Faulty Root" indicates instances where the root node of the signature was corrupted. In such cases, when signing the next layer's WOTS+, the attacker can obtain a usable faulty WOTS+ instance. Therefore, we consider these instances as successful fault injection, rather than merely incorrect signatures. The column "Faulty Signature, Root Correct" isolates cases where the signature was faulty despite the root node remaining uncorrupted. This scenario corresponds to a fault occurring before the WOTS+ signature of the current layer, leading

to a change in the position of a WOTS+ chain during signing, while the final WOTS+ public key remains correct. Across the five experiments, the average number of successful fault injection was 166, indicating a 16.2% probability of a successful attack when initiated at a random time during an XMSS signature at a single layer.

5.2 Comparison

Table 7: Comparison of Injection Success Rate and Attack Costs for SPHINCS+ with and without Protection

Parameter & Scheme		Phase 1 Injection Rate	Phase 2 Signature Queries	Phase 3 Forgery Probability	Storage	Layers
128s	SRCC & WOTS+CS (ours)	11.6%	2^{27}	$1/2^{25}$	16 MB	2
	WOTS+ Cache [Gen23]	11.6%	2^{27}	$1/2^{18}$	144 MB	2
	Unprotected [NIS24]	16.2%	2^{9}	$1/2^{18}$	0 MB	0
128f	SRCC & WOTS+CS (ours)	11.8%	2^{21}	$1/2^{25}$	18 MB	6
	WOTS+ Cache [Gen23]	11.8%	2^{21}	$1/2^{18}$	164 MB	6
	Unprotected [NIS24]	16.2%	2^{3}	$1/2^{18}$	0 MB	0
192s	SRCC & WOTS+CS (ours)	11.6%	2^{27}	$1/2^{37}$	24 MB	2
	WOTS+ Cache [Gen23]	11.6%	2^{27}	$1/2^{36}$	312 MB	2
	Unprotected [NIS24]	16.2%	2^{9}	$1/2^{36}$	0 MB	0
192f	SRCC & WOTS+CS (ours)	11.8%	2^{21}	$1/2^{37}$	27 MB	6
	WOTS+ Cache [Gen23]	11.8%	2^{21}	$1/2^{36}$	356 MB	6
	Unprotected [NIS24]	16.2%	2^{3}	$1/2^{36}$	0 MB	0
256s	SRCC & WOTS+CS (ours)	12.2%	2^{24}	$1/2^{50}$	8 MB	2
	WOTS+ Cache [Gen23]	12.2%	2^{24}	$1/2^{35}$	136 MB	2
	Unprotected [NIS24]	16.2%	2^{8}	$1/2^{35}$	0 MB	0
256f	SRCC & WOTS+CS (ours)	12.4%	2^{20}	$1/2^{50}$	8 MB	4
	WOTS+ Cache [Gen23]	12.4%	2^{20}	$1/2^{35}$	145 MB	4
	Unprotected [NIS24]	16.2%	2^{4}	$1/2^{35}$	0 MB	0

Table 7 compares the efficacy of our countermeasures against a random fault attack with the WOTS+ Cache countermeasures in [Gen23], across the three phases referencing those detailed in Subsection 3.1. The storage in Table 7 is calculated as $4n(2^{h'} + 2^{h' \cdot 2} + \cdots + 2^{h' \cdot c})$ bytes, where c denotes the number of cached layers, as described in Subsection 4.2.

For an unprotected system, the success probability of Phase 1 is 16.2%, derived from the experimental results discussed previously. With our protected scheme (SRCC & WOTS+CS), the Phase 1 success probability drops to 11.6%. This reduction occurs because SRCC stores the root nodes of all subtrees connected to the top few WOTS+ layers. In the 128-small variant, which has a total of 7 layers, this means only the remaining 5 layers are susceptible to successful attacks. Consequently, the probability becomes $16.2\% \times (5/7) \approx 11.6\%$.

In Phase 2, the attacker must find another signature corresponding to the faulty WOTS+ instance. In the worst-case scenario for an unprotected system, where the attacker targets the topmost WOTS+, they would on average need to query only $2^9 = 512$ signatures to encounter the same WOTS+ instance (given that each subtree layer in 128-small variant has a height of 9). However, with the SRCC protecting 2 layers at top, even in the worst case, an attacker can only target the third-highest layer of WOTS+. This increases the attacker's burden, requiring a minimum of $2^{27} = 134,217,728$ signature queries on average

to encounter the same WOTS+ instance. This substantial increase in required queries drastically raises the cost of attacks for the adversary.

5.3 Practical Security Assessment in Embedded Environments

A complete fault attack on SPHINCS+ is a multi-stage process requiring success in injection (Phase 1), signature acquisition (Phase 2), and signature forgery (Phase 3). Our proposed architecture implements defenses in all phases, where physical constraints and cryptographic enhancement reinforce each other across these phases.

Since SPHINCS+ is computationally intensive, generating signatures on embedded platforms like Cortex-M4 is slow. By forcing attackers to target lower layers in Phase 1, SRCC increases the required queries of signatures from an average of 2^3 (unprotected) to between 2^{20} and 2^{27} in Phase 2. While the caching countermeasure by Genêt [Gen23] also achieves this level of query complexity, our SRCC attains this delay with a reduced memory footprint (e.g., requiring 16 MB instead of 144 MB in the 128s setting). This increase transforms a sub-second attack into one requiring days or months of continuous operation. Such prolonged, high-frequency utilization of the cryptographic service triggers distinct anomalies that are easily detectable. In contrast, the negligible traffic of an unprotected attack (≈ 8 signatures) remains stealthy and practically instantaneous.

Even if an adversary overcomes the online bottleneck, they face the forgery phase (Phase 3). Our analysis revealed a bimodal distribution in standard WOTS+ post-reuse security, with a "left peak" as low as 35 bits. WOTS+CS eliminates this weakness, raising the security to 49.8 bits (a factor of 2^{15} improvement over the worst case). While 2^{50} computations are theoretically achievable offline, this cost—combined with the prohibitive online latency of Phase 2 renders the end-to-end attack impractical for practical embedded threat models.

6 Conclusion

In this paper, we presented a comprehensive, low-cost defense architecture for SPHINCS+ against fault attacks. We conducted a systematic analysis of the three-phase fault attack process, uncovering a critical vulnerability in WOTS+ post-reuse security: a bimodal probability distribution that lowers its average security. To mitigate this, we proposed WOTS+CS, a novel variant that modifies the checksum structure to eliminate this weakness and increase the difficulty of signature forgery. Concurrently, we introduced Subtree Root Children Caching (SRCC) as a countermeasure to the fault injection phase, which prevents the signing of incorrect subtree roots with minimal storage overhead and improved signing speed. Experiments validated that our hybrid approach of combining SRCC and WOTS+CS provides a defense, increasing attacker difficulty across all attack phases.

Thanks

We would like to sincerely thank the editors and reviewers for their valuable comments and suggestions. This work is supported by the National Natural Science Foundation of China (Grant No. 62272463).

References

[ABB+22] Jean-Philippe Aumasson, Daniel J. Bernstein, Ward Beullens, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Andreas Hülsing, Panos Kampanakis, Stefan Kölbl, Tanja Lange, Martin M. Lauridsen,

Florian Mendel, Ruben Niederhagen, Christian Rechberger, Joost Rijneveld, Peter Schwabe, and Bas Westerbaan. Sphincs+ submission to the nist post-quantum project, v.3.1. 2022.

[ALCZ20] Dorian Amiet, Lukas Leuenberger, Andreas Curiger, and Paul Zbinden. Fpga-based sphincs+ implementations: Mind the glitch. In *2020 23rd Euromicro Conference on Digital System Design (DSD)*, pages 229–237. IEEE, 2020.

[BDH11] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing. Xmss-a practical forward secure signature scheme based on minimal security assumptions. In *Post-Quantum Cryptography: 4th International Workshop, PQCrypto 2011, Taipei, Taiwan, November 29–December 2, 2011. Proceedings 4*, pages 117–129. Springer, 2011.

[BHH+15] Daniel J. Bernstein, Daira Hopwood, Andreas Hülsing, Tanja Lange, Ruben Niederhagen, Louiza Papachristodoulou, Michael Schneider, Peter Schwabe, and Zooko Wilcox-O'Hearn. Sphincs: Practical stateless hash-based signatures. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015*, pages 368–397, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.

[BHK+19] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS '19, page 2129–2146, New York, NY, USA, 2019. Association for Computing Machinery.

[CMP18] Laurent Castelnovi, Ange Martinelli, and Thomas Prest. Grafting trees: a fault attack against the sphincs framework. In *International Conference on Post-Quantum Cryptography*, pages 165–184. Springer, 2018.

[Flu23] Scott Fluhrer. Oops, i did it again revisited: another look at reusing one-time signatures. *Cryptology ePrint Archive*, 2023.

[GBH17] Leon Groot Bruinderink and Andreas Hülsing. "oops, i did it again"–security of one-time signatures under two-message attacks. In *International Conference on Selected Areas in Cryptography*, pages 299–322. Springer, 2017.

[Gen17] Aymeric Genêt. Hardware attacks against hash-based cryptographic algorithms. Technical report, Technical report, École polytechnique fédérale de Lausanne, 2017.

[Gen23] Aymeric Genêt. On protecting sphincs+ against fault attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(2):80–114, 2023.

[GOW+24] Ivan Gavrilan, Felix Oberhansl, Alexander Wagner, Emanuele Strieder, and Andreas Zankl. Impeccable keccak: Towards fault resilient sphincs+ implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):154–189, 2024.

[Gro96] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.

[Mer89] Ralph C Merkle. A certified digital signature. In *Conference on the Theory and Application of Cryptology*, pages 218–238. Springer, 1989.

[NIS16] CFP NIST. Submission requirements and evaluation criteria for the post-quantum cryptography standardization process, 2016.

[NIS24] CFP NIST. Stateless Hash-Based Digital Signature Standard. Technical Report FIPS 205, U.S. Department of Commerce, National Institute of Standards and Technology, Gaithersburg, MD 20899-8900, August 2024.

[Sho99] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.